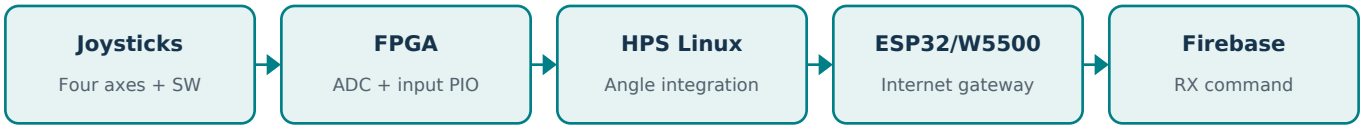
BUILD & OPERATE

DE10-Nano Robotic Arm Transmitter

Complete step-by-step guide and engineering record



Quartus + Platform Designer • Four ADC channels
Incremental joystick control • Claw toggle
Firebase • Python 2.7 • Persistent HTTPS

Document date: 27 October 2026
Includes editable source package and archived RBF

Recorded outcome: joystick inputs and TX-to-Firebase publishing verified. Includes source rebuild corrections, wiring and complete software. Rebuilt FPGA hardware validation and end-to-end latency remain outstanding.

Contents

Read this before building 3

1. Final system and required equipment 4

2. Physical connections and pin assignment 5

3. Memory map and control behaviour 7

4. Create the Quartus and Platform Designer project 8

5. HPS tabs and boot-image compatibility 9

6. Verilog, constraints and rebuild corrections 11

7. Boot Linux and load the TX FPGA image 13

8. Configure Internet and inspect joystick inputs 14

9. Firebase project, users, rules and data 15

10. Install the Python files through PuTTY 16

11. Run and operate the transmitter 17

12. Network implementation and measured results 18

13. Problems resolved and troubleshooting 19

14. Acceptance checklist and remaining limits 20

15. Sources and provenance 21

Appendix A. FPGA Verilog source 22

Appendix B. Complete TX Python software 27

Appendix C. Firebase configuration templates 33

Appendix D. Complete project configuration 34

Read this before building

This guide records the DE10-Nano transmitter configured on 27 October 2026. Its output is a Firebase command for the separate robotic-arm receiver. Four joystick axes control shoulder, base, elbow and wrist pitch; JS1's switch toggles the claw. Wrist roll is retained, not controlled by a joystick.

The verified endpoint is successful TX-to-Firebase operation. Physical TX-to-RX arm operation and complete motion latency were not measured in this session. A 140 ms median Firebase write is not a 140 ms guaranteed motor response.

Use the companion archive. It is attached inside this PDF as TX_Companion.zip and supplied separately. It contains complete Verilog, Qsys, QSF, generated synthesis files, Python source, parameter exports, checksums and the historical RBF. Long source lines may wrap visually in the PDF; install the original files from the archive when possible.

There are two FPGA directories. Original_FPGA is the user-identified Desktop project, preserved as evidence. Rebuild_FPGA contains explicitly documented corrections for a new compile. Those corrections were not loaded onto the user's working board during report preparation. Do not describe the rebuilt design as hardware-tested solely because the original board worked.

An existing, compatible DE10-Nano Linux SD-card image is required. This archive does not include the SD-card operating system, its bootloader image, or ESP32/W5500 router firmware. A blank SD card cannot boot from an RBF alone. The original HPS handoff has configuration discrepancies described in Chapter 5; it must not be used blindly to generate a new bootloader. A new reader must obtain a board-supported boot image and verify boot, Ethernet and DDR before continuing.

1. Final system and required equipment

1.1 Signal path

Two joysticks -> onboard LTC2308 ADC / FPGA button inputs -> FPGA input PIO registers -> lightweight HPS-to-FPGA bridge -> Python 2.7 TX controller -> persistent libcurl HTTPS requests -> Firebase /rx_arm/command -> separate RX controller and servo PWM.

The FPGA keeps raw ADC samples available. The HPS software performs calibration, dead-zone processing, angle integration, claw toggling, authentication and publishing. It never writes servo PWM registers on TX.

Item	Required configuration
FPGA board	Terasic DE10-Nano, Cyclone V 5CSEBA6U23I7
Joysticks	Two analog modules with VRx, VRy, SW, VCC and GND
Joystick supply	Regulated 3.3 V, common ground with the board
Development computer	Linux; Quartus Prime 25.1std / Lite with Cyclone V support
Serial console	PuTTY, USB-UART connection, 115200 baud, 8N1, no flow control
Board software observed	Angstrom v2014.12; kernel 4.5.0-00185-g3bb556b; Python 2.7.3
HTTPS runtime observed	curl 7.37.1, GnuTLS 3.3.5, libcurl.so.5; no pycurl installed
Internet	Ethernet from TX to an already configured ESP32/W5500 gateway, or laptop Ethernet sharing
Firebase	Realtime Database, Email/Password Authentication, separate TX writer and RX reader

1.2 Where commands run

Commands marked LAPTOP run in the Linux computer terminal. Commands marked TX/PuTTY run as root on the DE10-Nano. U-BOOT commands are entered only at the SOCFPGA_CYCLONE5 prompt before Linux boots. Firebase instructions refer to its web console.

Quartus project work occurs on the laptop. All board configuration in the verified TX session was performed through PuTTY. Do not paste Linux shell commands into U-Boot, or U-Boot commands into Linux.

2. Physical connections and pin assignment

2.1 Joystick wiring

Shut down TX with poweroff, disconnect board and joystick power, then wire. Use the board's printed A0-A3 labels to identify analog inputs. Do not count pins from a rotated photograph.

Module signal	TX connection	Function
JS1 VRx	Arduino A0 / ADC_IN0	Shoulder
JS1 VRy	Arduino A1 / ADC_IN1	Base
JS2 VRx	Arduino A2 / ADC_IN2	Elbow
JS2 VRy	Arduino A3 / ADC_IN3	Wrist pitch
JS1 SW	JP1 physical pin 1 / V12	Claw toggle
JS2 SW	Unconnected for current control scheme; archived input is JP1 pin 2 / E8	Unused
Both VCC pins	Regulated 3.3 V supply positive	Potentiometer excitation
Both GND pins	Supply negative and JP1 pin 12 or pin 30 GND	Common reference

Some joystick modules label VCC as +5V. Use 3.3 V here so the SW signal cannot pull an FPGA input to 5 V. If using an external 3.3 V supply, connect its ground to board ground but do not join its positive output to the board's own 3.3 V output. Keep the joystick supply off when the board is unpowered.

The original QSF enables FPGA weak pull-ups on JS1_SW and JS2_SW. A 10 kohm external pull-up from SW to 3.3 V can also be used when the module lacks one. The physical switch closes to ground. The FPGA debouncer inverts that active-low signal, so the register reports 0 released and 1 pressed. The final script learns the released bit during calibration instead of assuming polarity.

2.2 Board-level FPGA pins

These ADC signals are already wired on the PCB; do not connect joystick wipers to the FPGA's SPI pins.

HDL signal	FPGA package pin	Role
FPGA_CLK1_50	V11	50 MHz reference
ADC_CS_N	U9	Actually LTC2308 CONVST, active-high conversion start
ADC_SCLK	V10	ADC serial clock
ADC_SADDR	AC4	FPGA-to-ADC serial configuration
ADC_SDAT	AD4	ADC-to-FPGA serial sample
JS1_SW	V12	JP1 pin 1
JS2_SW	E8	JP1 pin 2

The archived QSF incorrectly assigns 1.2 V to the four ADC digital signals. Terasic's manual specifies 3.3 V for them. Rebuild_FPGA changes these four assignments to 3.3-V LVTTL. Analog input voltage, ADC reference voltage and FPGA digital I/O standards are separate concepts.

2.3 Legacy PWM pins present in the project

The historic top-level retains six local PWM outputs, although the TX Python program does not use them. Their vector is named GPIO_1, but these assignments physically land on JP1, not necessarily the header implied by that HDL name. Leave them unconnected on TX.

HDL signal	FPGA pin	JP1 physical pin
GPIO_1[0]	W12	3
GPIO_1[1]	D11	4
GPIO_1[2]	D8	5
GPIO_1[3]	AH13	6
GPIO_1[4]	AF7	7
GPIO_1[5]	AH14	8

Do not copy RX servo wiring onto the TX board. This report uses the original project port names so regeneration and source comparison remain possible.

3. Memory map and control behaviour

3.1 Lightweight bridge map

HPS physical base = 0xFF200000. The program maps 4096 bytes of /dev/mem as root and reads little-endian 32-bit words. The ADC PIOs carry 12 valid bits even though software reads a word.

PIO	Width / direction	Offset / CPU address
pio_pwm0..pio_pwm5	32-bit output, reset 0	0x00..0x50 at 0x10 spacing; unused by TX script
pio_js1_x	12-bit input	0x60 / 0xFF200060
pio_js1_y	12-bit input	0x70 / 0xFF200070
pio_js2_x	12-bit input	0x80 / 0xFF200080
pio_js2_y	12-bit input	0x90 / 0xFF200090
pio_buttons	2-bit input	0xA0 / 0xFF2000A0; bit 0 JS1, bit 1 JS2

These addresses are contractual. Do not let automatic address assignment move the inputs down to zero when removing unused PWM PIOs. This guide keeps all eleven PIOs to match the source project.

3.2 Incremental joystick control

The user requested position retention when the stick springs back. The final script interprets displacement as velocity, not absolute joint angle. At each iteration it adds velocity multiplied by elapsed time to the stored angle and clamps the result to 0..180 degrees.

At startup it samples each axis 100 times, roughly 10 ms apart, then uses the median as its centre. It rejects centres outside 500..2800 counts. Dead zone is 150 counts on each side. Maximum speed is 25 degrees/second; speed rises with deflection beyond the dead zone. The assumed upper raw endpoint is 3345, based on measured inputs. Recalibrate endpoints if using different joystick excitation or ADC scaling.

Positive means increasing ADC count. The physical direction depends on module orientation. To reverse one axis, multiply that axis's velocity by -1 in the integration line; test with motor power off first. At the centre, velocity is exactly zero and the target remains unchanged.

The claw switches between 0 and 180 on each debounced transition away from the learned released state. Software requires a stable new bit for at least 50 ms, in addition to the FPGA's approximately 20 ms debounce. Holding the switch does not repeatedly toggle. Endpoint meaning (open or closed) follows the RX mechanical calibration.

Roll angle and roll enable are copied from Firebase and preserved. In the logged final run roll was 135 degrees and disabled. Ignoring roll does not automatically disable a previously enabled roll channel.

4. Create the Quartus and Platform Designer project

4.1 Use the correct source set

The source provided by the user is /home/ooh/Desktop/de10nano_teleop. The similarly named directory under Desktop/Mega_Project contains an older empty ADC placeholder and is not suitable for reproduction. The older Board A PDF also contains a different register map and unrelated display/RF hardware; do not combine its top-level with this build.

For the most direct path, extract TX_Companion.zip, copy Rebuild_FPGA to a writable work folder, and open de10nano_teleop_top.qpf in Quartus. Original_FPGA is retained for comparison. Chapter 6 enumerates changes in the rebuilt copy.

To recreate the project through the GUI: File -> New Project Wizard; choose a new empty directory; set project and top-level entity to de10nano_teleop_top; select Cyclone V device 5CSEBA6U23I7. Add the four Verilog files from the companion. Open Tools -> Platform Designer and create soc_system.qsys in the project directory.

4.2 Clock and HPS

Add Clock Source named clk_0. Set frequency to 50,000,000 Hz, frequency known true, reset synchronous edges NONE. Export clk_in as clk and clk_in_reset as reset. Add Arria V/Cyclone V Hard Processor System named hps_0.

Enable lightweight HPS-to-FPGA bridge, 32 bits. Disable the full HPS-to-FPGA and FPGA-to-HPS bridges and direct FPGA-to-SDRAM ports. Disable unused debug/event/trace interfaces. Connect clk_0.clk to hps_0.h2f_lw_axi_clock. Export memory as memory and hps_io as hps_io.

The supplied saved Qsys does not export h2f_reset. The original top-level still named an unused h2f_reset_reset_n connection; this stale connection is removed in Rebuild_FPGA. Always compare the generated soc_system_inst.v template after generation.

4.3 Add the eleven PIOs

Add eleven PIO (Parallel I/O) components with the names and widths in Chapter 3. Input PIOs have direction Input; PWM PIOs have direction Output and reset value zero. Disable interrupt generation, edge capture and bit-modifying output registers. All clock inputs connect to clk_0.clk; all reset inputs connect to clk_0.clk_reset.

Connect every s1 slave to hps_0.h2f_lw_axi_master. Enter and lock the exact base addresses in Chapter 3. Export every external_connection with the name COMPONENT_external_connection, for example pio_js1_x_external_connection. The generated port adds _export to this exported interface name.

Validate the system, save it, then Generate HDL -> Verilog -> Synthesis. Add soc_system/synthesis/soc_system.qip to Quartus. Do not add every generated submodule manually or include duplicate generated systems through both copied and regenerated paths.

The archive includes reference/platform_parameters.csv, connections.csv and exports.csv, exported directly from the user's Qsys, for checking every stored value. These supplement the human-readable settings below.

5. HPS tabs and boot-image compatibility

5.1 Saved peripheral selections

This table records the actual TX Qsys. It is not a recommendation to create a new DE10-Nano preloader from these settings. The existing boot image initialized Ethernet independently; the live Linux log identifies its Ethernet controller at ff702000. The saved TX Qsys selects EMAC0, whereas the established RX design uses EMAC1. Reconcile the HPS configuration with the board-supported boot image before producing a new preloader/device tree.

Peripheral	Saved TX setting
EMAC0	HPS I/O Set 0; RGMII
EMAC1	Unused
SDIO	HPS I/O Set 0; 4-bit Data
UART0	HPS I/O Set 0; No Flow Control
USB0 / USB1	Unused
I2C0..3, SPI, NAND, QSPI, UART1	Unused

5.2 Saved HPS clocks

EOSC1 and EOSC2 are 25 MHz. FPGA-to-HPS reference-clock enables are off. Use-default-MPU-clock is enabled, desired MPU is 800 MHz, desired L4 MP and SP are 100 MHz, configuration clock is 100 MHz, desired SDMMC 200 MHz and desired EMAC clocks 250 MHz. The complete parameter export is authoritative for the saved project.

The live legacy SPL reports MPU 925 MHz and DDR 400 MHz. Loading an FPGA RBF does not replace those bootloader initialization settings. This is why a saved Qsys can differ from the running HPS environment without the software immediately failing.

5.3 Saved DDR settings - historical values, not board qualification

Parameter	Saved TX value
Protocol / vendor	DDR3 / JEDEC; device MISSING_MODEL
Memory clock / reference clock	300 MHz / 125 MHz
Total data width / DQS groups	32 / 4
Row / column / bank bits	15 / 10 / 3
Chip select / clock / CKE widths	1 / 1 / 1
Supply / CAS / write CAS	1.5 V DDR3 / 7 / 6
DM / DQS#	Enabled / enabled
Burst / read burst	OTF (4 or 8) / Sequential
Drive / nominal ODT / dynamic ODT	RZQ/6 / Disabled / Off
Self refresh / temperature	Manual / Normal

Saved memory timing: tIS 175 ps, tIH 250 ps, tDS 50 ps, tDH 125 ps, tDQSQ 120 ps, tDQSCK 400 ps, tQH 0.38 cycles, tDQSS 0.25 cycles, tQSH 0.38 cycles, tDSH and tDSS 0.2 cycles. tINIT 499 us; tMRD 3 cycles; tRAS 40 ns; tRCD and tRP 15 ns; tREFI 7 us; tRFC 75 ns; tWR 15 ns; tWTR 2 cycles; tFAW 37.5 ns; tRRD and tRTP 7.5 ns.

Board settings use automatic derating and automatic channel-integrity settings. Maximum CK and DQS delay are 0.6 ns; min/max CK-to-DQS skew -0.01/+0.01 ns; within/between-DQS skew 0.02 ns; address-command skew 0.02 ns; other eye reductions and average DQ/address offsets zero. Full values are in the CSV.

Do not use these historical generic DDR values to initialize a blank board. Use a DE10-Nano-supported boot image and matching board DDR preset/handoff. This report preserves the existing TX HDL for reproducibility, not a validated new HPS DDR initialization design. The operating-system image and matching bootloader are external prerequisites, not reconstructed from the project archive.

6. Verilog, constraints and rebuild corrections

6.1 Modules

The top-level instantiates soc_system, adc_controller, two debounce modules and six legacy pwm_generator instances. The ADC controller cycles CH0..CH3; its outputs connect directly to the four 12-bit input PIOs. The button PIO receives {js2_sw_clean, js1_sw_clean}.

The ADC controller advances a step counter once per ten 50 MHz clocks, a 200 ns tick. The SPI clock changes level on alternating transfer steps, so its active transfer frequency is approximately 2.5 MHz, not 5 MHz. The CONVST-high interval between step 0 and step 4 is approximately 0.8 us; the original comment says 1.0 us. The controller transmits commands 100010, 110010, 100110, 110110 and uses latched_ch to associate returned data with the previously selected channel. Preserve that pipeline when editing.

One 38-step transaction is approximately 7.6 us; a four-channel pass is approximately 30.4 us. These are logic-derived values, not oscilloscope measurements. Serial timing should be checked against the LTC2308 datasheet on a new hardware build.

6.2 Changes made only in Rebuild_FPGA

Change	Reason
Remove unused .h2f_reset_reset_n() port binding	Port absent from saved/generated Qsys interface
HPS_DDR3_ODT changed from 4 bits to scalar	Generated memory_mem_odt is one bit; avoids three unintended FPGA output pins
ADC digital I/O standards set to 3.3-V LVTTL	Matches Terasic board pin specification
Explicit Verilog source assignments and generated QIP	Avoid dependence on implicit file discovery
Explicit ODT D28 and RZQ D25 locations	Retain archived DDR pin mapping after narrowing ODT
Add tx_clock.sdc	Constrains 50 MHz FPGA clock with period 20 ns
Add power-on counter reset	Provides a short reset for ADC, system input reset and legacy PWM instead of tying them high

These changes are included for source rebuild and are not claimed to be present in the user's running historical RBF. Original_FPGA preserves the original versions. No new RBF was installed on the live TX during report creation.

The minimal SDC creates the 50 MHz clock and derives PLL clocks/uncertainty. It does not finish external ADC I/O timing closure. Review unconstrained paths and add datasheet-based ADC timing constraints for production qualification; do not suppress them with arbitrary false paths.

6.3 Compilation sequence (LAPTOP)

Use your Quartus installation path. From the copied Rebuild_FPGA directory:

```
quartus_map de10nano_teleop_top
quartus_sta -t soc_system/synthesis/submodules/hps_sdram_p0_pin_assignments.tcl de10nano_teleop_top
quartus_sh --flow compile de10nano_teleop_top
```

If Quartus is not on PATH, prefix each command with the installed quartus/bin directory. In this project it is /home/ooh/altera_lite/25.1std/quartus/bin/.

Inspect Analysis & Synthesis, Fitter and Timing Analyzer reports. Confirm ADC pins U9/V10/AC4/AD4, switches V12/E8 and clock V11. DDR pins and electrical constraints come from the generated HPS assignments; do not manually assign fixed HPS peripheral pins as arbitrary fabric pins.

Compilation success alone is insufficient: reject unexplained width mismatches, ignored locations, unassigned clock or missing ADC drivers. The archived September report contains a successful compile but warns that FPGA_CLK1_50 has no clock assignment. Its positive DDR slack is not proof that the complete original TX was timing-constrained.

6.4 Report-time build validation

Analysis and synthesis passed. After regenerating DDR assignments and explicitly assigning scalar ODT to D28 and RZQ to D25, Quartus Fitter passed on 27 October 2026 with 0 errors and 65 warnings. Every retained fitted signal matched the historical pin report (scalar ODT matched to old ODT[0]). The initial scalar-ODT placement failure was corrected in this package.

A warning still reports 70 pins without explicit user locations; the fitted mapping comparison is included in reference/pin_comparison.txt. Warnings about generated DDR clock uncertainty and external timing require review. This is synthesis/fitting evidence, not a board test or complete timing sign-off.

Timing Analyzer also completed with 0 errors and 61 warnings. For the slow 1100 mV / 100 C model, FPGA_CLK1_50 setup slack is 8.462 ns and hold slack is 0.319 ns. These are constrained internal-path results; external ADC timing and board validation remain required. New reports are in reference/rebuild.sta.summary and reference/rebuild_timing.log. No replacement RBF was generated or deployed during this validation.

6.5 Generate an RBF

In Quartus: File -> Convert Programming Files -> Raw Binary File (.rbf); select the appropriate passive-parallel configuration mode for the board's HPS-loading flow; add the compiled SOF; generate tx_soc.rbf. Retain the corresponding project sources and reports beside it. The historical supplied soc_system.rbf is 7,007,204 bytes; file size alone does not prove identity or correctness. Use SHA256SUMS.txt.

7. Boot Linux and load the TX FPGA image

7.1 Preserve the working boot environment

The recorded board already had a functioning Angstrom SD image and FPGA-loading boot sequence. U-Boot/SPL is 2013.01.01, built October 2016. SDRAM calibration passed and 1024 MiB was reported. A bad-environment-CRC warning appeared; do not assume saved environment variables persist.

For a new build, obtain a compatible DE10-Nano Linux SD image from the board supplier, write it to an identified spare card using the image's instructions, and boot it before changing FPGA files. Retain the original card. Set MSEL for the image's documented HPS FPGA-loading mode; this session did not establish the TX board's physical MSEL switch positions.

Connect the laptop to the board USB-UART connector, not the USB-Blaster programming connector. PuTTY: Serial, appropriate /dev/ttyUSB device, 115200, 8 data bits, 1 stop bit, no parity, no flow control. Enable All session output logging. Log in as root using the image's credentials.

7.2 Identify the boot file, then test a replacement

At U-Boot, first inspect rather than assume:

```
printenv bootcmd
printenv fpga
printenv fpgadata
printenv bridge_enable_handoff
fatls mmc 0:1
```

For the legacy flow, a candidate file on the FAT partition can be loaded using:

```
fatload mmc 0:1 0x02000000 tx_soc.rbf
fpga load 0 0x02000000 ${filesize}
run bridge_enable_handoff
run mmcload
run mmcboot
```

Use those run commands only if they exist in the inspected environment. Avoid a later boot step that reloads the old FPGA image over the candidate. Require successful FPGA load, Linux boot, Ethernet and changing ADC values. If the image uses different loading commands, follow its supplied boot script instead.

Only after this test, back up the old FPGA file and replace the exact filename referenced by the boot script (often soc_system.rbf). Verify the copy, sync, unmount and reboot. The running board accepts one active FPGA configuration at a time, although the FAT filesystem can store several RBF files. Do not overwrite an RX image or a bootloader file.

7.3 Basic inventory (TX/PuTTY)

```
uname -a
python --version
curl --version
ip addr show eth0
ip route
cat /etc/resolv.conf
date -u
```

The final application requires Python 2.7, ctypes, mmap, fcntl and libcurl. It uses /dev/mem and therefore runs as root. A modern Linux image may restrict /dev/mem; verify its supported FPGA bridge/driver workflow instead of assuming these legacy commands work unchanged.

8. Configure Internet and inspect joystick inputs

8.1 Gateway and addressing

The tested TX used an already configured ESP32 + W5500 Wi-Fi-to-Ethernet router. The board log showed default gateway 192.168.4.1 and primary address 192.168.4.50, with an additional secondary 192.168.4.2. Do not reproduce both addresses by default. Give every device on the same LAN a unique address; RX may already use 192.168.4.2.

Connect the TX RJ45 port to W5500 Ethernet and configure the router's upstream Wi-Fi using its existing firmware instructions. The firmware binary is not included. A laptop sharing its Wi-Fi through Ethernet is an alternative, but it normally uses a different subnet such as 10.42.0.0/24.

For an already working gateway at 192.168.4.1 and an unused TX address 192.168.4.50, the following is a temporary legacy configuration, not a persistent service definition:

```
ip link set eth0 up
ip addr replace 192.168.4.50/24 dev eth0
ip route replace default via 192.168.4.1 dev eth0
ping -c 3 192.168.4.1
ping -c 3 google.com
```

The observed DNS resolver was 8.8.8.8. Check DNS with nslookup and configure DNS through the image's network manager if needed. Do not overwrite a managed resolv.conf blindly. Internet addressing persistence depends on that manager and was not newly provisioned in this TX session.

8.2 Raw ADC diagnostic

Install test_adc.py from the software directory, or paste its complete listing in Appendix B using a quoted here-document. Run:

```
python -u /home/root/test_adc.py
```

Leave sticks centred, then move each axis separately to both ends. Press JS1 SW, release, repeat, then Ctrl+C. Expected: all four channels change, return near their own centre, and Buttons toggles 00/01 for JS1. The unused JS2 input must not be interpreted as a claw command.

Input	Observed range	Approximate rest reading
JS1 X	3..3342	1691
JS1 Y	4..3343	1654
JS2 X	2..3346	1722
JS2 Y	3..3346	1738

The first 483 samples had Buttons=00 throughout. A deliberate press/release test then produced 00/01 transitions. That established the switch path; it did not require changing the FPGA. Later calibration reported centres [1688,1652,1724,1740]. Do not hard-code 2048 as the centre for these 3.3 V-excited inputs.

9. Firebase project, users, rules and data

9.1 Create or reuse the project

For this build, project ID is robotic-arm-rx. Its database is https://robotic-arm-rx-default-rtdb.asia-southeast1.firebasedatabase.app and the command path is /rx_arm/command. A new builder should create their own project and use its URL/API key/users.

Firebase Console: create project -> Build -> Realtime Database -> Create Database -> choose an appropriate region -> start with locked rules. Authentication -> Sign-in method -> enable Email/Password. Create a TX writer user and a separate RX reader user. Copy their UIDs exactly.

Project Settings -> General -> Your apps -> register a Web app if needed -> copy apiKey from firebaseConfig. No JavaScript SDK is required on the board. The Python setup uses the Firebase Authentication REST endpoint and then passes its ID token to the database.

For historical reference, final TX UID was SnGoXE5FygZc9vGac5DgFWt3VNf1 and RX UID was Mfsh7prA78ZqW2oAAuVklFtZOkp1. Do not copy these into another project's rules. Deleting and recreating a user changes its UID.

9.2 Publish rules

Replace WRITER_UID and RX_UID with the actual UIDs before publishing:

```
{
  "rules": {
    ".read": false,
    ".write": false,
    "rx_arm": {
      "command": {
        ".read": "auth != null && (auth.uid === 'RX_UID' || auth.uid === 'WRITER_UID')",
        ".write": "auth != null && auth.uid === 'WRITER_UID'",

        ".validate": "newData.hasChildren(['seq', 'sent_ms', 'angles', 'enable'])"
      }
    }
  }
}
```

This is the existing minimal schema rule, not exhaustive type/range enforcement. RX also validates commands. TX needs read permission as well as write permission because it starts from stored targets.

9.3 Initialize the command

Import software/firebase_initial_data.json at the database root. It creates rx_arm/command with seq, sent_ms, all six angles and all six enable booleans. Initial enables are false. Set initial angles to a mechanically suitable pose for your arm before powering servos; an arbitrary 90-degree pose is not universally safe. The supplied neutral example uses 90 degrees for continuous joints and 0 for the claw, but is a schema example, not mechanical calibration.

On START, TX enables base, shoulder, elbow, pitch and claw automatically; roll enable is preserved. The script expects all six joints to exist, including unused roll. While TX is running, it owns the whole command object and overwrites manual console edits on the next publication. Stop TX before setting a new startup pose in Firebase.

10. Install the Python files through PuTTY

10.1 Files

Copy these source files from TX_Companion/software to /home/root on the board: tx_control.py, tx_http.py, tx_firebase_setup.py and test_adc.py. The final tx_control.py already contains the connection-reuse patch and the 20 ms pause. Do not reapply the old patch commands.

With PuTTY only, open each source file on the laptop, then enter this pattern on TX, paste the full file contents, and finish with PY on its own line:

```
cat > /home/root/tx_http.py <<'PY'
PASTE THE COMPLETE tx_http.py FILE CONTENTS HERE
PY
```

Repeat for the other files, changing the destination filename. The placeholder above is an instruction, not source code. Complete source listings are also in the appendices. If SSH is reachable, scp -O is an optional transfer method for old SSH servers; it is not required for the PuTTY-only procedure.

Compile-check on TX:

```
python -m py_compile /home/root/tx_control.py /home/root/tx_http.py /home/root/tx_firebase_setup.py
    /home/root/test_adc.py
```

Success returns to the shell silently. A syntax error usually means an incomplete paste, indentation damage or accidentally copied terminal prompts. Do not copy the > prompts or wrapped PuTTY log text back as source.

10.2 Set up authentication once

The supplied setup helper prompts for your database URL, Web API key, TX email and password. It signs in, reads /rx_arm/command and saves /home/root/tx_config.json only after both checks succeed. Password entry is hidden; the JSON file is plaintext on the board with mode 0600, not encrypted storage.

```
python /home/root/tx_firebase_setup.py
```

Expected: Sign-in HTTP 200; the correct Writer UID; Database read HTTP 200; Saved /home/root/tx_config.json. No command is written by setup. Credentials persist across reboots. Do not share the private JSON file or include it in a report.

The published helper is made reusable by prompting for project-specific fields; the live session helper originally hard-coded this project's public URL/key/email and later prompted for email. Authentication and permission-check behaviour are otherwise the same.

11. Run and operate the transmitter

11.1 First Firebase-only test

Keep RX motor power off for the first test, then run on TX:

```
python -u /home/root/tx_control.py
```

Release both sticks and switches, press Enter, and wait for calibration. Check the reported centres, starting targets and roll enable. Type START exactly. Observe /rx_arm/command/angles in Firebase while moving one axis at a time. Targets should increase/decrease with displacement, then stay constant after release. JS1 SW toggles the claw once per press.

Press Ctrl+C to stop. An in-flight write can complete during shutdown. RX holds its last received command; stopping TX does not disable the servos. Stop the RX program or use its validated disable procedure when you need PWM off.

11.2 Daily start

1. Power TX and its configured Internet gateway; let Linux boot.

2. Open PuTTY and log in as root. Verify Internet if the gateway has changed.

3. Run python -u /home/root/tx_control.py.

4. Release sticks/switches, press Enter, inspect the saved starting pose and type START.

5. Use the joysticks. No email/password entry is required unless credentials changed.

This is manual one-command startup with calibration prompts, not an installed auto-start service. The application and credentials persist on the root filesystem. An FPGA image must still be loaded by the existing boot process on every board restart.

11.3 RX integration

The paired RX must listen to the same database and /rx_arm/command schema. Its earlier /home/root/start_rx.py launcher used manual-console mode. That mode can consume the complete TX angle object, but bypasses timestamp freshness checks. Normal RX mode checks sequence and clock age; keep both clocks synchronized if switching to it.

Before powering motors, select a suitable startup pose with TX stopped. START enables the five controlled joints; old stored endpoints may cause a large movement. Test short movements and direction on each joint. This end-to-end motor test was not performed in the TX session recorded here, so it remains the final physical acceptance step for a new installation.

12. Network implementation and measured results

12.1 Why the transport changed

The first TX script started a new curl process for every request. Writes commonly took 630..1020 ms. The board had no pycurl package but did have libcurl.so.5, so tx_http.py now binds the library through ctypes. Each thread owns a persistent easy handle. Resetting its options preserves reusable connections and DNS/session caches. TLS peer and hostname verification remain enabled.

The main thread samples/integrates joystick targets. A separate publisher copies the latest targets under a lock and performs one synchronous PUT. There is no queue of every sampled position. The publisher waits 20 ms after each successful response. With a 140 ms request this means roughly a 160 ms publication cycle, about 6 updates/second, not 50 updates/second.

The token is renewed by signing in again after about 3000 seconds or after a failed request. Requests have 5-second connection and 8-second total timeouts; DNS timeout behaviour can depend on how the old libcurl resolver was built. On detected errors the script pauses target integration and retries with up to a 5-second wait. While an already-started request is blocked, integration may continue until that failure is detected. This is not an immediate link-loss interlock or a hardware watchdog.

12.2 Observed performance

Metric	Final logged test
Successful write samples	478
Minimum request time	105 ms
Median request time	140 ms
95th percentile (rank method)	165 ms
Maximum, including first connection	766 ms
Wrist roll	135 degrees, disabled, preserved

request_ms measures HTTP PUT duration on TX. It excludes the time before a new target is captured for a request and does not measure RX processing, PWM-frame alignment or mechanical movement. Do not claim guaranteed 50..150 ms end-to-end latency from these values. First connections and Internet/Wi-Fi jitter can be slower.

The final log shows values holding across many writes and the claw toggling. Small base changes occurred while other movements were being made; the suggested additional untouched-stick test was not completed. A new builder should run that 10-second rest test and adjust dead zone only if drift is actually observed at rest.

13. Problems resolved and troubleshooting

Symptom	Finding / action
Invalid login credentials	Use the Firebase Email/Password user's password, not an assumed Gmail password; verify email and project
Sign-in 200, curl exit 6	DNS failure for database hostname; inspect resolver and nslookup; retry after resolution works
Sign-in 200, database HTTP 401	Setup initially used /arm/command; corrected to /rx_arm/command; keep locked rules
Read succeeds but write fails	Verify TX UID matches writer rule and full schema is sent
Buttons stays 00	Confirm physical click, SW wiring and pull-up; deliberate test showed 01 pressed
Raw centre not 2048	Measured centres near 1650..1740; median calibration and dead zone handle this
Large Firebase delay	Replace per-request subprocess with persistent libcurl handles
curl exit -2 on Ctrl+C	Original curl child interrupted by shutdown; not an authentication fault
ImportError: tx_http	Put tx_http.py beside tx_control.py in /home/root
libcurl error 60	Check board time and CA trust; do not bypass TLS verification
GPIO reads frozen or bus access hangs	Check loaded bitstream, bridge state and offsets before further /dev/mem access
Lock acquisition fails	Another tx_control process may already be running; stop the intended process, do not run two publishers
Endpoint jumps at startup	TX reads stored targets, not actual motor angles; set a suitable pose before START
Fresh project will not elaborate	Use corrected top-level, regenerate Qsys and compare generated interface names

Only one transmitter should publish this command object at a time. The local flock prevents duplicate copies of this script on one board, but cannot stop another computer or Firebase-console editor from writing the same path.

14. Acceptance checklist and remaining limits

1. Board boots a supported Linux image, DDR calibration succeeds, and the intended FPGA image loads.

2. Rebuilt project has the correct pin assignments and no unexplained synthesis/fitter errors; review external-interface timing separately.

3. Four ADC channels change independently; switches report the expected bits.

4. Released-stick calibration passes and targets remain constant during a 10-second untouched test.

5. Firebase sign-in and read succeed for the correct TX UID; writes update /rx_arm/command.

6. Roll remains unchanged and each claw press produces one toggle.

7. Reboot TX and verify normal start using saved credentials and the intended FPGA image.

8. With a suitable initial pose, complete physical TX-to-RX movement testing and measure end-to-end latency if required.

This is an open-loop command system: TX reads no encoder or measured servo position. Network loss leaves RX holding its last target, and the current software has no independent emergency-stop watchdog. The archived source and newer reproduction corrections have distinct validation status. Hardware revision, SD image, preloader, network and mechanical calibration remain installation dependencies.

15. Sources and provenance

Primary project: user-identified /home/ooh/Desktop/de10nano_teleop. The software listings were reconstructed from the issued commands and PuTTY transcript, with terminal wrapping removed and final patches applied. The log was renamed by the user to /home/ooh/Desktop/tx_putty27Oct.log during report preparation.

The private tx_config.json and passwords are excluded. Original_FPGA includes the historical soc_system.rbf and synthesis source; Rebuild_FPGA includes only the clearly identified reproduction corrections. CSV exports provide all saved component parameters and connections. SHA256SUMS.txt covers the companion contents.

[S1] Terasic DE10-Nano User Manual, especially GPIO/ADC pin tables and boot configuration: https://fpgacademy.org/Downloads/DE10_Nano_User_Manual.pdf

[S2] Firebase Authentication REST API: https://firebase.google.com/docs/reference/rest/auth/

[S3] Firebase REST authentication: https://firebase.google.com/docs/database/rest/auth

[S4] libcurl reset and connection preservation: https://curl.se/libcurl/c/curl_easy_reset.html

[S5] libcurl threading rules: https://curl.se/libcurl/c/threadsafe.html

[S6] libcurl receive callback: https://curl.se/libcurl/c/CURLOPT_WRITEFUNCTION.html

The original Board A document and the older Mega_Project source were reviewed but not used as the authority for this TX's address map or FPGA top-level. Their differences are explicitly called out to prevent mixing designs.

Appendix A. FPGA Verilog source

Use the matching files in the archive; visual line wrapping in this PDF is not a source-code newline.

Rebuild_FPGA/de10nano_teleop_top.v

```
// Top-level module for the DE10-Nano 2-Joystick -> 6-DOF Robotic Arm Teleop project
module de10nano_teleop_top (
    input  wire        FPGA_CLK1_50,

    // HPS DDR3
    output wire [14:0] HPS_DDR3_ADDR,
    output wire [2:0]  HPS_DDR3_BA,
    output wire        HPS_DDR3_CK_P,

    output wire        HPS_DDR3_CK_N,
    output wire        HPS_DDR3_CKE,
    output wire        HPS_DDR3_CS_N,
    output wire        HPS_DDR3_RAS_N,
    output wire        HPS_DDR3_CAS_N,
    output wire        HPS_DDR3_WE_N,
    output wire        HPS_DDR3_RESET_N,
    inout  wire [31:0] HPS_DDR3_DQ,
    inout  wire [3:0]  HPS_DDR3_DQS_P,
    inout  wire [3:0]  HPS_DDR3_DQS_N,
    output wire        HPS_DDR3_ODT,
    output wire [3:0]  HPS_DDR3_DM,
    input  wire        HPS_DDR3_RZQ,

    // HPS Ethernet (EMAC0 / RGMII)
    output wire        HPS_ENET_GTX_CLK,
    output wire [3:0]  HPS_ENET_TX_DATA,
    input  wire [3:0]  HPS_ENET_RX_DATA,
    inout  wire        HPS_ENET_MDIO,
    output wire        HPS_ENET_MDC,
    input  wire        HPS_ENET_RX_DV,
    output wire        HPS_ENET_TX_EN,
    input  wire        HPS_ENET_RX_CLK,

    // HPS SD Card (SDIO, 4-bit)
    inout  wire        HPS_SD_CMD,
    inout  wire [3:0]  HPS_SD_DATA,
    output wire        HPS_SD_CLK,

    // HPS UART0
    input  wire        HPS_UART_RX,
    output wire        HPS_UART_TX,

    // Onboard LTC2308 ADC
    output wire        ADC_CS_N,
    output wire        ADC_SADDR,
    output wire        ADC_SCLK,
    input  wire        ADC_SDAT,

    // Joystick Pushbuttons (active-low with pull-ups)
    input  wire        JS1_SW,
    input  wire        JS2_SW,

    // Servo PWM Outputs -> GPIO 1 header [0: Base, 1: Shoulder, 2: Elbow, 3: Pitch, 4: Roll, 5: Gripper]
    output wire [5:0]  GPIO_1
);

    // Added for reproducible startup; original tied resets high.
    reg [5:0] power_on_count = 0;
    always @(posedge FPGA_CLK1_50)
        if (!power_on_count[5]) power_on_count <= power_on_count + 1'b1;
    wire power_on_reset_n = power_on_count[5];

    // Internal Wires
    wire [11:0] js1_x, js1_y, js2_x, js2_y;
    wire        js1_sw_clean, js2_sw_clean;
    wire [31:0] pwm_duty [0:5];

    // 1. Platform Designer System: HPS (Memory + HPS IO) + 11 PIOs
    soc_system u0 (
        .clk_clk                            (FPGA_CLK1_50),
        .reset_reset_n                      (power_on_reset_n),
```

```
        // DDR3 SDRAM
        .memory_mem_a                       (HPS_DDR3_ADDR),
        .memory_mem_ba                      (HPS_DDR3_BA),
        .memory_mem_ck                      (HPS_DDR3_CK_P),
        .memory_mem_ck_n                    (HPS_DDR3_CK_N),
        .memory_mem_cke                     (HPS_DDR3_CKE),
        .memory_mem_cs_n                    (HPS_DDR3_CS_N),
        .memory_mem_ras_n                   (HPS_DDR3_RAS_N),
        .memory_mem_cas_n                   (HPS_DDR3_CAS_N),
        .memory_mem_we_n                    (HPS_DDR3_WE_N),
        .memory_mem_reset_n                 (HPS_DDR3_RESET_N),
        .memory_mem_dq                      (HPS_DDR3_DQ),
        .memory_mem_dqs                     (HPS_DDR3_DQS_P),
        .memory_mem_dqs_n                   (HPS_DDR3_DQS_N),
        .memory_mem_odt                     (HPS_DDR3_ODT),
        .memory_mem_dm                      (HPS_DDR3_DM),
        .memory_oct_rzqin                   (HPS_DDR3_RZQ),

        // Ethernet
        .hps_io_hps_io_emac0_inst_TX_CLK    (HPS_ENET_GTX_CLK),
        .hps_io_hps_io_emac0_inst_TXD0      (HPS_ENET_TX_DATA[0]),
        .hps_io_hps_io_emac0_inst_TXD1      (HPS_ENET_TX_DATA[1]),
        .hps_io_hps_io_emac0_inst_TXD2      (HPS_ENET_TX_DATA[2]),
        .hps_io_hps_io_emac0_inst_TXD3      (HPS_ENET_TX_DATA[3]),
        .hps_io_hps_io_emac0_inst_RXD0      (HPS_ENET_RX_DATA[0]),
        .hps_io_hps_io_emac0_inst_MDIO      (HPS_ENET_MDIO),
        .hps_io_hps_io_emac0_inst_MDC       (HPS_ENET_MDC),
        .hps_io_hps_io_emac0_inst_RX_CTL    (HPS_ENET_RX_DV),
        .hps_io_hps_io_emac0_inst_TX_CTL    (HPS_ENET_TX_EN),
        .hps_io_hps_io_emac0_inst_RX_CLK    (HPS_ENET_RX_CLK),
        .hps_io_hps_io_emac0_inst_RXD1      (HPS_ENET_RX_DATA[1]),
        .hps_io_hps_io_emac0_inst_RXD2      (HPS_ENET_RX_DATA[2]),
        .hps_io_hps_io_emac0_inst_RXD3      (HPS_ENET_RX_DATA[3]),

        // SD Card
        .hps_io_hps_io_sdio_inst_CMD        (HPS_SD_CMD),
        .hps_io_hps_io_sdio_inst_D0         (HPS_SD_DATA[0]),
        .hps_io_hps_io_sdio_inst_D1         (HPS_SD_DATA[1]),
        .hps_io_hps_io_sdio_inst_CLK        (HPS_SD_CLK),
        .hps_io_hps_io_sdio_inst_D2         (HPS_SD_DATA[2]),
        .hps_io_hps_io_sdio_inst_D3         (HPS_SD_DATA[3]),

        // UART0
        .hps_io_hps_io_uart0_inst_RX        (HPS_UART_RX),
        .hps_io_hps_io_uart0_inst_TX        (HPS_UART_TX),

        // Joystick Inputs (Inputs to HPS)
        .pio_buttons_external_connection_export ({js2_sw_clean, js1_sw_clean}),
        .pio_js1_x_external_connection_export   (js1_x),
        .pio_js1_y_external_connection_export   (js1_y),
        .pio_js2_x_external_connection_export   (js2_x),
        .pio_js2_y_external_connection_export   (js2_y),

        // Servo PWM Outputs (Outputs from HPS)
        .pio_pwm0_external_connection_export    (pwm_duty[0]),
        .pio_pwm1_external_connection_export    (pwm_duty[1]),
        .pio_pwm2_external_connection_export    (pwm_duty[2]),
        .pio_pwm3_external_connection_export    (pwm_duty[3]),
        .pio_pwm4_external_connection_export    (pwm_duty[4]),
        .pio_pwm5_external_connection_export    (pwm_duty[5])
    );

    // 2. ADC Controller: Reads LTC2308 and feeds joystick channels
    adc_controller u_adc (
        .clk      (FPGA_CLK1_50),
        .reset_n  (power_on_reset_n),
        .adc_cs_n (ADC_CS_N),
        .adc_saddr(ADC_SADDR),
        .adc_sclk (ADC_SCLK),
        .adc_sdat (ADC_SDAT),
        .ch0_data (js1_x),
        .ch1_data (js1_y),
        .ch2_data (js2_x),
        .ch3_data (js2_y)
    );

    // 3. Debounce Blocks for Joystick Switches
    debounce u_db1 (
        .clk       (FPGA_CLK1_50),
        .noisy_in  (JS1_SW),
```

```
        .clean_out (js1_sw_clean)
    );

    debounce u_db2 (
        .clk       (FPGA_CLK1_50),
        .noisy_in  (JS2_SW),
        .clean_out (js2_sw_clean)
    );

    // 4. Six PWM Generators (one per servo)
    genvar i;
    generate
        for (i = 0; i < 6; i = i + 1) begin : pwm_gen
            pwm_generator #(
                .CLK_FREQ_HZ(50_000_000)
            ) u_pwm (
                .clk               (FPGA_CLK1_50),
                .reset_n           (power_on_reset_n),
                .pulse_width_counts (pwm_duty[i]),
                .pwm_out           (GPIO_1[i])
            );
        end
    endgenerate

endmodule
```

Rebuild_FPGA/adc_controller.v

```
module adc_controller (
    input  wire        clk,        // 50 MHz Clock
    input  wire        reset_n,    // Active-low reset
    output reg         adc_cs_n,   // ADC_CONVST
    output reg         adc_saddr,  // ADC_SDI
    output reg         adc_sclk,   // ADC_SCK
    input  wire        adc_sdat,   // ADC_SDO
    output reg  [11:0] ch0_data,   // Joystick 1 X

    output reg  [11:0] ch1_data,   // Joystick 1 Y
    output reg  [11:0] ch2_data,   // Joystick 2 X
    output reg  [11:0] ch3_data    // Joystick 2 Y
);

    // 5 MHz SPI tick generator (50 MHz / 10 = 200 ns period)
    reg [3:0] clk_div;
    reg       tick;

    always @(posedge clk or negedge reset_n) begin
        if (!reset_n) begin
            clk_div <= 4'd0;
            tick    <= 1'b0;
        end else begin
            if (clk_div == 4'd9) begin
                clk_div <= 4'd0;
                tick    <= 1'b1;
            end else begin
                clk_div <= clk_div + 1'b1;
                tick    <= 1'b0;
            end
        end
    end

    // LTC2308 6-bit channel commands: {S/D, O/S, S1, S0, UNI, SLP}
    // Single-ended, unipolar: CH0=100010, CH1=110010, CH2=100110, CH3=110110
    reg [5:0] cmd_table [0:3];
    initial begin
        cmd_table[0] = 6'b100010; // CH0
        cmd_table[1] = 6'b110010; // CH1
        cmd_table[2] = 6'b100110; // CH2
        cmd_table[3] = 6'b110110; // CH3
    end

    reg [6:0]  step;
    reg [1:0]  cur_ch;
    reg [1:0]  latched_ch;
    reg [5:0]  cmd_shift;
    reg [11:0] rx_shift;

    always @(posedge clk or negedge reset_n) begin
        if (!reset_n) begin
            adc_cs_n   <= 1'b0;
```

```
            adc_sclk   <= 1'b0;
            adc_saddr  <= 1'b0;
            ch0_data   <= 12'd2048;
            ch1_data   <= 12'd2048;
            ch2_data   <= 12'd2048;
            ch3_data   <= 12'd2048;
            step       <= 7'd0;
            cur_ch     <= 2'd0;
            latched_ch <= 2'd0;
            cmd_shift  <= 6'd0;
            rx_shift   <= 12'd0;
        end else if (tick) begin
            case (step)
                // 1. Send CONVST pulse (active high for 1.0 us)
                7'd0: begin
                    adc_cs_n  <= 1'b1;
                    adc_sclk  <= 1'b0;
                    cmd_shift <= cmd_table[cur_ch];
                    step      <= step + 1'b1;
                end
                7'd1, 7'd2, 7'd3: step <= step + 1'b1;
                7'd4: begin
                    adc_cs_n <= 1'b0;
                    step     <= step + 1'b1;
                end

                // 2. Wait t_CONV (1.6 us)
                7'd5, 7'd6, 7'd7, 7'd8, 7'd9, 7'd10, 7'd11, 7'd12: step <= step + 1'b1;

                // 3. Shift 12 bits: transmit configuration command and read channel sample
                7'd13, 7'd15, 7'd17, 7'd19, 7'd21, 7'd23,
                7'd25, 7'd27, 7'd29, 7'd31, 7'd33, 7'd35: begin
                    adc_sclk  <= 1'b0;
                    adc_saddr <= cmd_shift[5];
                    cmd_shift <= {cmd_shift[4:0], 1'b0};
                    step      <= step + 1'b1;
                end

                7'd14, 7'd16, 7'd18, 7'd20, 7'd22, 7'd24,
                7'd26, 7'd28, 7'd30, 7'd32, 7'd34, 7'd36: begin
                    adc_sclk <= 1'b1;
                    rx_shift <= {rx_shift[10:0], adc_sdat};
                    step     <= step + 1'b1;
                end

                // 4. Latch 12-bit value into the respective register
                7'd37: begin
                    adc_sclk <= 1'b0;
                    case (latched_ch)
                        2'd0: ch0_data <= rx_shift;
                        2'd1: ch1_data <= rx_shift;
                        2'd2: ch2_data <= rx_shift;
                        2'd3: ch3_data <= rx_shift;
                    endcase
                    latched_ch <= cur_ch;
                    cur_ch     <= cur_ch + 1'b1;
                    step       <= 7'd0;
                end

                default: step <= 7'd0;
            endcase
        end
    end

endmodule
```

Rebuild_FPGA/debounce.v

```
module debounce #(
    parameter CLK_FREQ_HZ   = 50_000_000,
    parameter STABLE_TIME_MS = 20
)(
    input  wire clk,
    input  wire noisy_in,
    output reg  clean_out
);

    localparam integer SAMPLE_DIVIDE = CLK_FREQ_HZ / 1000;
    localparam integer STABLE_COUNT  = STABLE_TIME_MS;
```

```
    reg [$clog2(SAMPLE_DIVIDE)-1:0] sample_clk_div = 0;
    reg sample_tick = 0;
    reg [$clog2(STABLE_COUNT+1)-1:0] stable_counter = 0;
    reg sync_0, sync_1;
    reg last_sampled;

    initial clean_out = 1'b0;

    always @(posedge clk) begin
        sync_0 <= noisy_in;
        sync_1 <= sync_0;
    end

    always @(posedge clk) begin
        if (sample_clk_div == SAMPLE_DIVIDE - 1) begin
            sample_clk_div <= 0;
            sample_tick    <= 1'b1;
        end else begin
            sample_clk_div <= sample_clk_div + 1'b1;
            sample_tick    <= 1'b0;
        end
    end

    always @(posedge clk) begin
        if (sample_tick) begin
            if (sync_1 == last_sampled) begin
                if (stable_counter < STABLE_COUNT)
                    stable_counter <= stable_counter + 1'b1;
                else
                    clean_out <= ~sync_1; // Inverts active-low joystick buttons
            end else begin
                stable_counter <= 0;
            end
            last_sampled <= sync_1;
        end
    end
endmodule
```

Rebuild_FPGA/pwm_generator.v

```
module pwm_generator #(
    parameter CLK_FREQ_HZ = 50_000_000
)(
    input  wire        clk,
    input  wire        reset_n,
    input  wire [31:0] pulse_width_counts,
    output reg         pwm_out
);

    localparam FRAME_COUNTS = CLK_FREQ_HZ / 50; // 20ms period (50 Hz frame)

    reg [19:0] counter;

    always @(posedge clk or negedge reset_n) begin
        if (!reset_n) begin
            counter <= 0;
            pwm_out <= 0;
        end else begin
            counter <= (counter >= FRAME_COUNTS - 1) ? 0 : counter + 1;
            pwm_out <= (counter < pulse_width_counts);
        end
    end
endmodule
```

Rebuild_FPGA/tx_clock.sdc

```
create_clock -name FPGA_CLK1_50 -period 20.000 [get_ports {FPGA_CLK1_50}]
derive_pll_clocks
derive_clock_uncertainty
```

Appendix B. Complete TX Python software

Use the matching files in the archive; visual line wrapping in this PDF is not a source-code newline.

software/tx_control.py

```
from __future__ import print_function
import os
import sys
import json
import time
import mmap
import struct
import subprocess

import threading
import fcntl

JOINTS = ('base', 'shoulder', 'elbow', 'pitch', 'roll', 'claw')
AXES = ('shoulder', 'base', 'elbow', 'pitch')
OFFSETS = (0x60, 0x70, 0x80, 0x90)
SPEED = 25.0
DEADZONE = 150.0

def quote(s):
    return '"' + s.replace('\\', '\\\\').replace('"', '\\"').replace('\n', '\\n').replace('\r', '\\r') + '"'

def request(url, payload=None, method=None):
    cfg = [
        'url = ' + quote(url),
        'silent',
        'ipv4',
        'connect-timeout = 5',
        'max-time = 8',
        'write-out = "\\n%{http_code}"'
    ]
    if method:
        cfg.append('request = ' + quote(method))
    if payload is not None:
        cfg += [
            'header = "Content-Type: application/json"',
            'data = ' + quote(json.dumps(payload))
        ]
    p = subprocess.Popen(
        ['curl', '--config', '-'],
        stdin=subprocess.PIPE,
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE
    )
    out, err = p.communicate(('\n'.join(cfg) + '\n').encode('utf-8'))
    if p.returncode:
        raise RuntimeError('curl exit %d' % p.returncode)
    body, status = out.decode('utf-8').rsplit('\n', 1)
    if int(status) != 200:
        raise RuntimeError('Firebase HTTP ' + status)
    return json.loads(body)

def velocity(raw, centre):
    d = raw - centre
    if abs(d) <= DEADZONE:
        return 0.0
    span = (3345 - centre) if d > 0 else centre
    direction = 1 if d > 0 else -1
    fraction = (abs(d) - DEADZONE) / max(1, span - DEADZONE)
    return direction * SPEED * min(1.0, fraction)

def main():
    lockfile = open('/tmp/tx_control.lock', 'a')
    fcntl.flock(lockfile, fcntl.LOCK_EX | fcntl.LOCK_NB)

    with open('/home/root/tx_config.json') as f:
        config = json.load(f)

    def login():
        result = request(
            'https://identitytoolkit.googleapis.com/v1/accounts:signInWithPassword?key='
            + config['api_key'],
```

```
        dict(email=config['email'], password=config['password'],
             returnSecureToken=True)
    )
    return result['idToken']

token = login()
endpoint = config['database_url'].rstrip('/') + '/rx_arm/command.json?auth='
state = request(endpoint + token)
if not isinstance(state, dict):
    raise ValueError('Set initial RX targets in Firebase first')

angles = state['angles']
enables = state['enable']
for joint in JOINTS:
    value = angles[joint]
    if isinstance(value, bool) or not isinstance(value, (int, long, float)):
        raise ValueError('Invalid angle: ' + joint)
    if not 0 <= value <= 180:
        raise ValueError('Angle outside 0..180: ' + joint)
    if type(enables[joint]) is not bool:
        raise ValueError('Invalid enable: ' + joint)

sequence = [max(int(state.get('seq', 0)), int(time.time() * 1000000))]

fd = os.open('/dev/mem', os.O_RDWR | os.O_SYNC)
mem = mmap.mmap(fd, 4096, mmap.MAP_SHARED,
                mmap.PROT_READ, offset=0xFF200000)
os.close(fd)

def read(offset):
    return struct.unpack_from('<I', mem, offset)[0]

raw_input('Release both sticks and switches; press Enter to calibrate: ')
samples = []
for n in range(100):
    samples.append([read(o) for o in OFFSETS] + [read(0xA0) & 1])
    time.sleep(.01)

centres = [sorted(s[i] for s in samples)[50] for i in range(4)]
if any(c < 500 or c > 2800 for c in centres):
    raise ValueError('Centre out of range; check joystick wiring')

idle = samples[-1][4]
if any(s[4] != idle for s in samples):
    raise ValueError('Switch changed during calibration; restart')

print('Centres:', centres)
print('Starting targets:', angles)
print('Roll unchanged:', angles['roll'], 'enabled:', enables['roll'])

if raw_input('Type START to enable the five controlled joints: ').strip() != 'START':
    mem.close()
    return

for joint in AXES + ('claw',):
    enables[joint] = True

mutex = threading.Lock()
stop = threading.Event()
connected = threading.Event()

def publish():
    auth = token
    renewed = time.time()
    failures = 0
    while not stop.is_set():
        try:
            if time.time() - renewed > 3000 or failures:
                auth = login()
                renewed = time.time()
            with mutex:
                packet = dict(
                    angles=dict(angles),
                    enable=dict(enables),
                    seq=sequence[0],
                    sent_ms=int(time.time() * 1000)
                )
            begin = time.time()
            request(endpoint + auth, packet, 'PUT')
            failures = 0
```

```
                connected.set()
                print('Sent request_ms=%.0f angles=%s' %
                      (1000 * (time.time() - begin), packet['angles']))
                sys.stdout.flush()
                stop.wait(.02)
            except Exception as error:
                connected.clear()
                failures += 1
                print('Connection issue:', str(error))
                print('Targets paused; RX holds its last received command.')
                sys.stdout.flush()
                stop.wait(min(5, failures))

    worker = threading.Thread(target=publish)
    worker.daemon = True
    worker.start()

    previous = os.times()[4]
    candidate = idle
    stable = idle
    changed = previous

    try:
        while True:
            now = os.times()[4]
            dt = min(.05, max(0, now - previous))
            previous = now
            raw = [read(o) for o in OFFSETS]
            bit = read(0xA0) & 1

            if bit != candidate:
                candidate = bit
                changed = now

            pressed = False
            if candidate != stable and now - changed >= .05:
                stable = candidate
                pressed = stable != idle

            with mutex:
                if connected.is_set():
                    for i, joint in enumerate(AXES):
                        target = angles[joint] + velocity(raw[i], centres[i]) * dt
                        angles[joint] = round(max(0, min(180, target)), 4)
                    if pressed:
                        angles['claw'] = 0.0 if angles['claw'] >= 90 else 180.0
                sequence[0] += 1
            time.sleep(.01)

    except KeyboardInterrupt:
        print('\nStopping TX; RX retains its last received targets.')
    finally:
        stop.set()
        worker.join(18)
        mem.close()

from tx_http import request

if __name__ == '__main__':
    try:
        main()
    except Exception as error:
        print('Stopped:', str(error))
        sys.exit(1)
```

software/tx_http.py

```
import ctypes as C
import ctypes.util
import threading
import json

lib = C.CDLL(ctypes.util.find_library('curl'))
P = C.c_void_p
lib.curl_global_init.argtypes = [C.c_long]

lib.curl_global_init.restype = C.c_int
lib.curl_easy_init.argtypes = []
lib.curl_easy_init.restype = P
lib.curl_easy_setopt.argtypes = [P, C.c_int]
```

```
lib.curl_easy_setopt.restype = C.c_int
lib.curl_easy_getinfo.argtypes = [P, C.c_int]
lib.curl_easy_getinfo.restype = C.c_int
lib.curl_easy_perform.argtypes = [P]
lib.curl_easy_perform.restype = C.c_int
lib.curl_easy_reset.argtypes = [P]
lib.curl_easy_reset.restype = None
lib.curl_slist_append.argtypes = [P, C.c_char_p]
lib.curl_slist_append.restype = P
lib.curl_slist_free_all.argtypes = [P]
lib.curl_slist_free_all.restype = None

if lib.curl_global_init(3):
    raise RuntimeError('libcurl initialization failed')

local = threading.local()
Callback = C.CFUNCTYPE(
    C.c_size_t, P, C.c_size_t, C.c_size_t, P
)

def request(url, payload=None, method=None):
    if not hasattr(local, 'handle'):
        local.handle = lib.curl_easy_init()
        if not local.handle:
            raise RuntimeError('Cannot allocate curl handle')
    h = local.handle
    lib.curl_easy_reset(h)
    parts = []

    @Callback
    def receive(ptr, size, count, userdata):
        length = size * count
        try:
            parts.append(C.string_at(ptr, length))
            return length
        except Exception:
            return 0

    def option(key, value):
        rc = lib.curl_easy_setopt(h, key, value)
        if rc:
            raise RuntimeError('curl option %d error %d' % (key, rc))

    headers = None
    try:
        url_bytes = url.encode('utf-8')
        option(10002, C.c_char_p(url_bytes))
        option(20011, receive)

        # No signals; IPv4; timeouts; HTTP/1.1; verify TLS.
        for key, value in (
            (99, 1), (113, 1), (156, 5000), (155, 8000),
            (84, 2), (64, 1), (81, 2), (52, 0)
        ):
            option(key, C.c_long(value))

        if payload is not None:
            body = json.dumps(
                payload, separators=(',', ':')
            ).encode('utf-8')
            headers = lib.curl_slist_append(
                None, b'Content-Type: application/json'
            )
            if not headers:
                raise RuntimeError('Cannot allocate headers')
            extra = lib.curl_slist_append(headers, b'Expect:')
            if not extra:
                raise RuntimeError('Cannot allocate headers')
            headers = extra
            option(10023, P(headers))
            option(60, C.c_long(len(body)))
            option(10015, C.c_char_p(body))

        if method:
            method_bytes = method.encode('ascii')
            option(10036, C.c_char_p(method_bytes))

        rc = lib.curl_easy_perform(h)
        if rc:
            raise RuntimeError('libcurl error %d' % rc)
```

```
        status = C.c_long()
        rc = lib.curl_easy_getinfo(
            h, 0x200002, C.byref(status)
        )
        if rc:
            raise RuntimeError('Cannot read HTTP status')
        if status.value != 200:
            raise RuntimeError('Firebase HTTP %d' % status.value)
        return json.loads(b''.join(parts).decode('utf-8'))
    finally:
        # Reset options while retaining cached connections.
        lib.curl_easy_reset(h)
        if headers:
            lib.curl_slist_free_all(headers)
```

software/tx_firebase_setup.py

```
from __future__ import print_function
import json
import os
import getpass
import subprocess

def quote(s):
    return '"' + s.replace('\\', '\\\\').replace('"', '\\"').replace('\n', '\\n').replace('\r', '\\r') + '"'

def request(url, payload=None):
    cfg = [
        'url = ' + quote(url),
        'silent',
        'connect-timeout = 10',
        'max-time = 30',
        'write-out = "\\n%{http_code}"'
    ]
    if payload is not None:
        cfg += [
            'header = "Content-Type: application/json"',
            'data = ' + quote(json.dumps(payload))
        ]
    p = subprocess.Popen(
        ['curl', '--config', '-'],
        stdin=subprocess.PIPE,
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE
    )
    out, err = p.communicate(('\n'.join(cfg) + '\n').encode('utf-8'))
    if p.returncode:
        raise RuntimeError('Network/TLS error; curl exit %s' % p.returncode)
    body, status = out.decode('utf-8').rsplit('\n', 1)
    return int(status), json.loads(body)

c = {}

try:
    c['database_url'] = raw_input('Firebase database URL (https://...): ').strip().rstrip('/')
    c['api_key'] = raw_input('Firebase Web API key: ').strip()
    c['email'] = raw_input('Firebase TX email: ').strip()
    c['password'] = getpass.getpass('Firebase TX account password: ')
    status, result = request(
        'https://identitytoolkit.googleapis.com/v1/accounts:signInWithPassword?key=' + c['api_key'],
        {'email': c['email'], 'password': c['password'],
         'returnSecureToken': True}
    )
    print('Sign-in HTTP:', status)
    if status != 200:
        print('Firebase error:', result.get('error'))
        raise SystemExit(1)

    print('Writer UID:', result['localId'])
    status, command = request(
        c['database_url'] + '/rx_arm/command.json?auth=' + result['idToken']
    )
    print('Database read HTTP:', status)
    if status != 200:
        print('Database error:', command)
        raise SystemExit(1)

    os.umask(0o077)
    path = '/home/root/tx_config.json'
    with open(path, 'w') as f:
```

```
        json.dump(c, f, indent=2)
    os.chmod(path, 0o600)
    print('Saved:', path)
    print('Firebase connection OK. No commands were written.')
except Exception as e:
    print('Setup failed:', str(e))
```

software/test_adc.py

```
from __future__ import print_function
import os, mmap, struct, time
fd = os.open('/dev/mem', os.O_RDWR | os.O_SYNC)
m = mmap.mmap(fd, 4096, mmap.MAP_SHARED,
              mmap.PROT_READ | mmap.PROT_WRITE, offset=0xff200000)
try:
    while True:
        values = [struct.unpack_from('<I', m, o)[0] for o in (0x60,0x70,0x80,0x90,0xa0)]

        print('JS1 X=%d Y=%d | JS2 X=%d Y=%d | buttons=%02x' % tuple(values))
        time.sleep(0.1)
except KeyboardInterrupt:
    pass
finally:
    m.close()
    os.close(fd)
```

Appendix C. Firebase configuration templates

Use the matching files in the archive; visual line wrapping in this PDF is not a source-code newline.

software/database.rules.example.json

```
{
  "rules": {
    ".read": false,
    ".write": false,
    "rx_arm": {
      "command": {
        ".read": "auth != null && (auth.uid === 'RX_UID' || auth.uid === 'WRITER_UID')",
        ".write": "auth != null && auth.uid === 'WRITER_UID'",

        ".validate": "newData.hasChildren(['seq', 'sent_ms', 'angles', 'enable'])"
      }
    }
  }
}
```

software/initial_database.json

```
{
  "rx_arm": {
    "command": {
      "seq": 0,
      "sent_ms": 0,
      "angles": [
        90,
        90,

        90,
        90,
        90,
        0
      ],
      "enable": [
        false,
        false,
        false,
        false,
        false,
        false
      ]
    }
  }
}
```

software/tx_config.example.json

```
{
  "database_url": "https://YOUR-DATABASE.firebasedatabase.app",
  "api_key": "YOUR_WEB_API_KEY",
  "email": "YOUR_TX_EMAIL",
  "password": "YOUR_TX_PASSWORD"
}
```

Appendix D. Complete project configuration

The archive includes the complete Qsys XML and generated HDL. reference/platform_parameters.csv, connections.csv and exports.csv retain every saved parameter and connection. The following QSF is the corrected build configuration, including generated DDR constraints.

```
# -------------------------------------------------------------------------- #
#
# Copyright (C) 2025  Altera Corporation. All rights reserved.
# Your use of Altera Corporation's design tools, logic functions
# and other software and tools, and any partner logic
# functions, and any output files from any of the foregoing
# (including device programming or simulation files), and any
# associated documentation or information are expressly subject

# to the terms and conditions of the Altera Program License
# Subscription Agreement, the Altera Quartus Prime License Agreement,
# the Altera IP License Agreement, or other applicable license
# agreement, including, without limitation, that your use is for
# the sole purpose of programming logic devices manufactured by
# Altera and sold by Altera or its authorized distributors.  Please
# refer to the Altera Software License Subscription Agreements
# on the Quartus Prime software download page.
#
# -------------------------------------------------------------------------- #
#
# Quartus Prime
# Version 25.1std.0 Build 1129 10/21/2025 SC Lite Edition
# Date created = 22:19:34  September 06, 2026
#
# -------------------------------------------------------------------------- #
#
# Notes:
#
# 1) The default values for assignments are stored in the file:
#□□de10nano_teleop_top_assignment_defaults.qdf
#    If this file doesn't exist, see file:
#□□assignment_defaults.qdf
#
# 2) Intel recommends that you do not modify this file. This
#    file is updated automatically by the Quartus Prime software
#    and any changes you make may be lost or overwritten.
#
# -------------------------------------------------------------------------- #


set_global_assignment -name FAMILY "Cyclone V"
set_global_assignment -name DEVICE 5CSEBA6U23I7
set_global_assignment -name TOP_LEVEL_ENTITY de10nano_teleop_top
set_global_assignment -name ORIGINAL_QUARTUS_VERSION 25.1STD.0
set_global_assignment -name PROJECT_CREATION_TIME_DATE "22:19:34  SEPTEMBER 06, 2026"
set_global_assignment -name LAST_QUARTUS_VERSION "25.1std.0 Lite Edition"
set_global_assignment -name PROJECT_OUTPUT_DIRECTORY output_files
set_global_assignment -name MIN_CORE_JUNCTION_TEMP "-40"
set_global_assignment -name MAX_CORE_JUNCTION_TEMP 100
set_global_assignment -name ERROR_CHECK_FREQUENCY_DIVISOR 256
set_global_assignment -name EDA_SIMULATION_TOOL "Questa Altera FPGA (Verilog)"
set_global_assignment -name EDA_TIME_SCALE "1 ps" -section_id eda_simulation
set_global_assignment -name EDA_OUTPUT_DATA_FORMAT "VERILOG HDL" -section_id eda_simulation
set_global_assignment -name EDA_GENERATE_FUNCTIONAL_NETLIST OFF -section_id eda_board_design_timing
set_global_assignment -name EDA_GENERATE_FUNCTIONAL_NETLIST OFF -section_id eda_board_design_symbol
set_global_assignment -name EDA_GENERATE_FUNCTIONAL_NETLIST OFF -section_id eda_board_design_signal_integrity
set_global_assignment -name EDA_GENERATE_FUNCTIONAL_NETLIST OFF -section_id eda_board_design_boundary_scan
set_global_assignment -name QIP_FILE soc_system/synthesis/soc_system.qip
set_global_assignment -name POWER_PRESET_COOLING_SOLUTION "23 MM HEAT SINK WITH 200 LFPM AIRFLOW"
set_global_assignment -name POWER_BOARD_THERMAL_MODEL "NONE (CONSERVATIVE)"
# 50 MHz System Clock
set_location_assignment PIN_V11 -to FPGA_CLK1_50
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to FPGA_CLK1_50

# LTC2308 ADC
set_location_assignment PIN_U9 -to ADC_CS_N
set_location_assignment PIN_V10 -to ADC_SCLK
set_location_assignment PIN_AC4 -to ADC_SADDR
set_location_assignment PIN_AD4 -to ADC_SDAT
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to ADC_CS_N
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to ADC_SCLK
```

```
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to ADC_SADDR
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to ADC_SDAT

# Joystick Pushbuttons (Header JP1: Pins 1 & 2)
set_location_assignment PIN_V12 -to JS1_SW
set_location_assignment PIN_E8 -to JS2_SW
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to JS*
set_instance_assignment -name WEAK_PULL_UP_RESISTOR ON -to JS*

# 6-DOF Servos (Header JP1: Pins 3, 4, 5, 6, 7, 8)
# [0] Base, [1] Shoulder, [2] Elbow, [3] Wrist Pitch, [4] Wrist Roll, [5] Gripper
set_location_assignment PIN_W12 -to GPIO_1[0]
set_location_assignment PIN_D11 -to GPIO_1[1]
set_location_assignment PIN_D8 -to GPIO_1[2]
set_location_assignment PIN_AH13 -to GPIO_1[3]
set_location_assignment PIN_AF7 -to GPIO_1[4]
set_location_assignment PIN_AH14 -to GPIO_1[5]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to GPIO_1[*]
set_global_assignment -name PARTITION_NETLIST_TYPE SOURCE -section_id Top
set_global_assignment -name PARTITION_FITTER_PRESERVATION_LEVEL PLACEMENT_AND_ROUTING -section_id Top
set_global_assignment -name PARTITION_COLOR 16764057 -section_id Top
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_RZQ -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[0] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[0] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[0] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[1] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[1] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[1] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[2] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[2] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[2] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[3] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[3] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[3] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[4] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[4] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[4] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[5] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[5] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[5] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[6] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[6] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[6] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[7] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[7] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[7] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[8] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[8] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[8] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[9] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[9] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[9] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[10] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[10] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[10] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[11] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[11] -tag
    __hps_sdram_p0
```

```
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[11] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[12] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[12] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[12] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[13] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[13] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[13] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[14] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[14] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[14] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[15] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[15] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[15] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[16] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[16] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[16] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[17] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[17] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[17] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[18] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[18] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[18] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[19] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[19] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[19] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[20] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[20] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[20] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[21] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[21] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[21] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[22] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[22] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[22] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[23] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[23] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[23] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[24] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[24] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[24] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[25] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[25] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[25] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[26] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[26] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[26] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[27] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[27] -tag
    __hps_sdram_p0
```

```
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[27] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[28] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[28] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[28] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[29] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[29] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[29] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[30] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[30] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[30] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DQ[31] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[31] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQ[31] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "DIFFERENTIAL 1.5-V SSTL CLASS I" -to HPS_DDR3_DQS_P[0] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQS_P[0] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQS_P[0] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "DIFFERENTIAL 1.5-V SSTL CLASS I" -to HPS_DDR3_DQS_P[1] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQS_P[1] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQS_P[1] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "DIFFERENTIAL 1.5-V SSTL CLASS I" -to HPS_DDR3_DQS_P[2] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQS_P[2] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQS_P[2] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "DIFFERENTIAL 1.5-V SSTL CLASS I" -to HPS_DDR3_DQS_P[3] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQS_P[3] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQS_P[3] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "DIFFERENTIAL 1.5-V SSTL CLASS I" -to HPS_DDR3_DQS_N[0] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQS_N[0] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQS_N[0] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "DIFFERENTIAL 1.5-V SSTL CLASS I" -to HPS_DDR3_DQS_N[1] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQS_N[1] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQS_N[1] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "DIFFERENTIAL 1.5-V SSTL CLASS I" -to HPS_DDR3_DQS_N[2] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQS_N[2] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQS_N[2] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "DIFFERENTIAL 1.5-V SSTL CLASS I" -to HPS_DDR3_DQS_N[3] -tag __hps_sdram_p0
set_instance_assignment -name INPUT_TERMINATION "PARALLEL 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQS_N[3] -tag
    __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DQS_N[3] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "DIFFERENTIAL 1.5-V SSTL CLASS I" -to HPS_DDR3_CK_P -tag __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITHOUT CALIBRATION" -to HPS_DDR3_CK_P -tag
    __hps_sdram_p0
set_instance_assignment -name D5_DELAY 2 -to HPS_DDR3_CK_P -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "DIFFERENTIAL 1.5-V SSTL CLASS I" -to HPS_DDR3_CK_N -tag __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITHOUT CALIBRATION" -to HPS_DDR3_CK_N -tag
    __hps_sdram_p0
set_instance_assignment -name D5_DELAY 2 -to HPS_DDR3_CK_N -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_ADDR[0] -tag __hps_sdram_p0
set_instance_assignment -name CURRENT_STRENGTH_NEW "MAXIMUM CURRENT" -to HPS_DDR3_ADDR[0] -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_ADDR[10] -tag __hps_sdram_p0
set_instance_assignment -name CURRENT_STRENGTH_NEW "MAXIMUM CURRENT" -to HPS_DDR3_ADDR[10] -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_ADDR[11] -tag __hps_sdram_p0
set_instance_assignment -name CURRENT_STRENGTH_NEW "MAXIMUM CURRENT" -to HPS_DDR3_ADDR[11] -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_ADDR[12] -tag __hps_sdram_p0
set_instance_assignment -name CURRENT_STRENGTH_NEW "MAXIMUM CURRENT" -to HPS_DDR3_ADDR[12] -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_ADDR[13] -tag __hps_sdram_p0
set_instance_assignment -name CURRENT_STRENGTH_NEW "MAXIMUM CURRENT" -to HPS_DDR3_ADDR[13] -tag __hps_sdram_p0
```

```
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_ADDR[14] -tag __hps_sdram_p0
set_instance_assignment -name CURRENT_STRENGTH_NEW "MAXIMUM CURRENT" -to HPS_DDR3_ADDR[14] -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_ADDR[1] -tag __hps_sdram_p0
set_instance_assignment -name CURRENT_STRENGTH_NEW "MAXIMUM CURRENT" -to HPS_DDR3_ADDR[1] -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_ADDR[2] -tag __hps_sdram_p0
set_instance_assignment -name CURRENT_STRENGTH_NEW "MAXIMUM CURRENT" -to HPS_DDR3_ADDR[2] -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_ADDR[3] -tag __hps_sdram_p0
set_instance_assignment -name CURRENT_STRENGTH_NEW "MAXIMUM CURRENT" -to HPS_DDR3_ADDR[3] -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_ADDR[4] -tag __hps_sdram_p0
set_instance_assignment -name CURRENT_STRENGTH_NEW "MAXIMUM CURRENT" -to HPS_DDR3_ADDR[4] -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_ADDR[5] -tag __hps_sdram_p0
set_instance_assignment -name CURRENT_STRENGTH_NEW "MAXIMUM CURRENT" -to HPS_DDR3_ADDR[5] -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_ADDR[6] -tag __hps_sdram_p0
set_instance_assignment -name CURRENT_STRENGTH_NEW "MAXIMUM CURRENT" -to HPS_DDR3_ADDR[6] -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_ADDR[7] -tag __hps_sdram_p0
set_instance_assignment -name CURRENT_STRENGTH_NEW "MAXIMUM CURRENT" -to HPS_DDR3_ADDR[7] -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_ADDR[8] -tag __hps_sdram_p0
set_instance_assignment -name CURRENT_STRENGTH_NEW "MAXIMUM CURRENT" -to HPS_DDR3_ADDR[8] -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_ADDR[9] -tag __hps_sdram_p0
set_instance_assignment -name CURRENT_STRENGTH_NEW "MAXIMUM CURRENT" -to HPS_DDR3_ADDR[9] -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_BA[0] -tag __hps_sdram_p0
set_instance_assignment -name CURRENT_STRENGTH_NEW "MAXIMUM CURRENT" -to HPS_DDR3_BA[0] -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_BA[1] -tag __hps_sdram_p0
set_instance_assignment -name CURRENT_STRENGTH_NEW "MAXIMUM CURRENT" -to HPS_DDR3_BA[1] -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_BA[2] -tag __hps_sdram_p0
set_instance_assignment -name CURRENT_STRENGTH_NEW "MAXIMUM CURRENT" -to HPS_DDR3_BA[2] -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_CAS_N -tag __hps_sdram_p0
set_instance_assignment -name CURRENT_STRENGTH_NEW "MAXIMUM CURRENT" -to HPS_DDR3_CAS_N -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_CKE -tag __hps_sdram_p0
set_instance_assignment -name CURRENT_STRENGTH_NEW "MAXIMUM CURRENT" -to HPS_DDR3_CKE -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_CS_N -tag __hps_sdram_p0
set_instance_assignment -name CURRENT_STRENGTH_NEW "MAXIMUM CURRENT" -to HPS_DDR3_CS_N -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_RAS_N -tag __hps_sdram_p0
set_instance_assignment -name CURRENT_STRENGTH_NEW "MAXIMUM CURRENT" -to HPS_DDR3_RAS_N -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_WE_N -tag __hps_sdram_p0
set_instance_assignment -name CURRENT_STRENGTH_NEW "MAXIMUM CURRENT" -to HPS_DDR3_WE_N -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_RESET_N -tag __hps_sdram_p0
set_instance_assignment -name CURRENT_STRENGTH_NEW "MAXIMUM CURRENT" -to HPS_DDR3_RESET_N -tag __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DM[0] -tag __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DM[0] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DM[1] -tag __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DM[1] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DM[2] -tag __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DM[2] -tag
    __hps_sdram_p0
set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_DM[3] -tag __hps_sdram_p0
set_instance_assignment -name OUTPUT_TERMINATION "SERIES 50 OHM WITH CALIBRATION" -to HPS_DDR3_DM[3] -tag
    __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[0] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[1] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[2] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[3] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[4] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[5] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[6] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[7] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[8] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[9] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[10] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[11] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[12] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[13] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[14] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[15] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[16] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[17] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[18] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[19] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[20] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[21] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[22] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[23] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[24] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[25] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[26] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[27] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[28] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[29] -tag __hps_sdram_p0
```

```
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[30] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQ[31] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DM[0] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DM[1] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DM[2] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DM[3] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQS_P[0] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQS_P[1] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQS_P[2] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQS_P[3] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQS_N[0] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQS_N[1] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQS_N[2] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_DQS_N[3] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_ADDR[0] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_ADDR[10] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_ADDR[11] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_ADDR[12] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_ADDR[13] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_ADDR[14] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_ADDR[1] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_ADDR[2] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_ADDR[3] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_ADDR[4] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_ADDR[5] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_ADDR[6] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_ADDR[7] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_ADDR[8] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_ADDR[9] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_BA[0] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_BA[1] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_BA[2] -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_CAS_N -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_CKE -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_CS_N -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_RAS_N -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_WE_N -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_RESET_N -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_CK_P -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_CK_N -tag __hps_sdram_p0
set_instance_assignment -name GLOBAL_SIGNAL OFF -to
    u0|hps_0|hps_io|border|hps_sdram_inst|p0|umemphy|ureset|phy_reset_mem_stable_n -tag __hps_sdram_p0
set_instance_assignment -name GLOBAL_SIGNAL OFF -to u0|hps_0|hps_io|border|hps_sdram_inst|p0|umemphy|ureset|phy_reset_n
     -tag __hps_sdram_p0
set_instance_assignment -name GLOBAL_SIGNAL OFF -to
    u0|hps_0|hps_io|border|hps_sdram_inst|p0|umemphy|uio_pads|dq_ddio[0].read_capture_clk_buffer -tag __hps_sdram_p0
set_instance_assignment -name GLOBAL_SIGNAL OFF -to
    u0|hps_0|hps_io|border|hps_sdram_inst|p0|umemphy|uread_datapath|reset_n_fifo_write_side[0] -tag __hps_sdram_p0
set_instance_assignment -name GLOBAL_SIGNAL OFF -to
    u0|hps_0|hps_io|border|hps_sdram_inst|p0|umemphy|uread_datapath|reset_n_fifo_wraddress[0] -tag __hps_sdram_p0
set_instance_assignment -name GLOBAL_SIGNAL OFF -to
    u0|hps_0|hps_io|border|hps_sdram_inst|p0|umemphy|uio_pads|dq_ddio[1].read_capture_clk_buffer -tag __hps_sdram_p0
set_instance_assignment -name GLOBAL_SIGNAL OFF -to
    u0|hps_0|hps_io|border|hps_sdram_inst|p0|umemphy|uread_datapath|reset_n_fifo_write_side[1] -tag __hps_sdram_p0
set_instance_assignment -name GLOBAL_SIGNAL OFF -to
    u0|hps_0|hps_io|border|hps_sdram_inst|p0|umemphy|uread_datapath|reset_n_fifo_wraddress[1] -tag __hps_sdram_p0
set_instance_assignment -name GLOBAL_SIGNAL OFF -to
    u0|hps_0|hps_io|border|hps_sdram_inst|p0|umemphy|uio_pads|dq_ddio[2].read_capture_clk_buffer -tag __hps_sdram_p0
set_instance_assignment -name GLOBAL_SIGNAL OFF -to
    u0|hps_0|hps_io|border|hps_sdram_inst|p0|umemphy|uread_datapath|reset_n_fifo_write_side[2] -tag __hps_sdram_p0
set_instance_assignment -name GLOBAL_SIGNAL OFF -to
    u0|hps_0|hps_io|border|hps_sdram_inst|p0|umemphy|uread_datapath|reset_n_fifo_wraddress[2] -tag __hps_sdram_p0
set_instance_assignment -name GLOBAL_SIGNAL OFF -to
    u0|hps_0|hps_io|border|hps_sdram_inst|p0|umemphy|uio_pads|dq_ddio[3].read_capture_clk_buffer -tag __hps_sdram_p0
set_instance_assignment -name GLOBAL_SIGNAL OFF -to
    u0|hps_0|hps_io|border|hps_sdram_inst|p0|umemphy|uread_datapath|reset_n_fifo_write_side[3] -tag __hps_sdram_p0
set_instance_assignment -name GLOBAL_SIGNAL OFF -to
    u0|hps_0|hps_io|border|hps_sdram_inst|p0|umemphy|uread_datapath|reset_n_fifo_wraddress[3] -tag __hps_sdram_p0
set_instance_assignment -name ENABLE_BENEFICIAL_SKEW_OPTIMIZATION_FOR_NON_GLOBAL_CLOCKS ON -to
    u0|hps_0|hps_io|border|hps_sdram_inst -tag __hps_sdram_p0
set_instance_assignment -name PLL_COMPENSATION_MODE DIRECT -to u0|hps_0|hps_io|border|hps_sdram_inst|pll0|fbout -tag
    __hps_sdram_p0
set_global_assignment -name USE_DLL_FREQUENCY_FOR_DQS_DELAY_CHAIN ON
set_global_assignment -name UNIPHY_SEQUENCER_DQS_CONFIG_ENABLE ON
set_global_assignment -name OPTIMIZE_MULTI_CORNER_TIMING ON
set_global_assignment -name ECO_REGENERATE_REPORT ON
set_global_assignment -name NUM_PARALLEL_PROCESSORS ALL
set_instance_assignment -name PARTITION_HIERARCHY root_partition -to | -section_id Top
set_global_assignment -name VERILOG_FILE de10nano_teleop_top.v
set_global_assignment -name VERILOG_FILE adc_controller.v
```

```
set_global_assignment -name VERILOG_FILE debounce.v
set_global_assignment -name VERILOG_FILE pwm_generator.v
set_global_assignment -name SDC_FILE tx_clock.sdc

set_instance_assignment -name IO_STANDARD "SSTL-15 CLASS I" -to HPS_DDR3_ODT -tag __hps_sdram_p0
set_instance_assignment -name CURRENT_STRENGTH_NEW "MAXIMUM CURRENT" -to HPS_DDR3_ODT -tag __hps_sdram_p0
set_instance_assignment -name PACKAGE_SKEW_COMPENSATION OFF -to HPS_DDR3_ODT -tag __hps_sdram_p0
set_location_assignment PIN_D28 -to HPS_DDR3_ODT
set_location_assignment PIN_D25 -to HPS_DDR3_RZQ
```